

EE-133L PROGRAMMING FUNDAMENTALS

Laboratory Manual



Student Name: _____

Registration No. _____

Lab Section: _____

**Department of Electrical Engineering
University of Engineering and Technology, Lahore**



Programming Fundamentals Lab Manual



Lab Safety and Discipline Rules for Programming Fundamentals Laboratories

1. Students must not work alone in the lab during official hours; they should be supervised by an instructor or lab staff.
2. Always log in to the computer with your assigned credentials. Do not attempt to access unauthorized accounts or systems.
3. Before running your program, carefully check your code, logic, and syntax. Discuss with your partner/classmate if needed and re-check before executing. If unsure, ask your lab instructor.
4. Any modification in your code should be properly saved with version control (different filenames or commits) to avoid accidental data loss.
5. Avoid experimenting with harmful code (e.g., infinite loops, virus scripts, system commands) that may damage the computer or affect other users.
6. After completing the experiment, properly save your work, log out from your account, and shut down/restart the system as instructed.
7. Do not attempt to install or uninstall software on lab computers. If any software issue arises, report it to the lab staff/instructor.
8. Do not use damaged peripherals (mouse, keyboard, power cord, USB) or computers with overheating/faults. Report such issues immediately.
9. When testing or debugging, carefully observe error messages and avoid forceful restarts or shutdowns. If the system hangs, inform the instructor.
10. Use USB drives or cloud storage responsibly. Do not connect unknown/unverified devices to lab computers.
11. Students should wear proper attire in the computer lab. Slippers are not recommended; maintain a professional environment.
12. Do not bring or consume food and beverages inside the computer lab.
13. Avoid working on computers with wet hands or while carrying liquids nearby.
14. Remove loose jewelry, accessories, and keep long hair tied back to prevent interference with fans or moving parts of machines.
15. Always ensure the computer and software you are using are within their allowed capacity (e.g., do not overload the system with unnecessary programs).
16. Never overload the computer by running multiple heavy applications simultaneously.
17. Do not attempt to bypass security settings, firewalls, or antivirus systems.
18. Ensure all external devices (USB, portable drives) are properly scanned and safely removed after use.
19. Backup your code and data regularly to avoid data loss.

INSTRUCTORS:	Ms. Khush Bakht Ms. Bareera Sadia		SESSION:	2026		SEMESTER:	2 TH (Spring-2026)	
SUBJECT TITLE:	Programming Fundamentals Lab							
LAB PERFORMANCE EVALUATION RUBRIC								
Performance	CLOs Mapping	Excellent (100%–80%)	Good (80%–55%)	Satisfactory (55%–30%)	Needs Improvement (30%–0%)			
Logic / Approach to Problem Solving (30 Points)	CLO1, CLO2, CLO3.	Demonstrates clear, innovative, and structured problem-solving approach, selects the most efficient logic and justifies choices effectively.	Applies appropriate logic with minor gaps, solution works with some inefficiency but shows understanding of approach.	Basic attempt to apply logic but with significant gaps, solution is partially correct and lacks depth.	Unable to apply a logical approach, solution is unclear, incorrect, or missing.			
Code Implementation & Error Debugging (40 Points)	CLO1, CLO2, CLO3.	Code is well-structured, follows best practices, and compiles successfully on the first attempt; independently debugs errors with precision.	Code is mostly correct with minor issues; requires some debugging but resolves independently.	Code contains multiple errors; requires assistance of lab instructor for debugging and corrections.	Code is incomplete, incorrect, or fails to run; unable to debug or resolve errors even with help.			
Documentation & Report Writing (20 Points)	CLO4	Documentation is comprehensive, professional, and well-formatted, includes clear explanations, comments, and structured reporting.	Documentation is complete with minor formatting or clarity issues; comments are present but limited.	Documentation is minimal, lacks clarity, or inconsistent formatting; explanations are insufficient.	Documentation/report is missing, unclear, or unprofessional.			
Discipline & Lab Ethics (10 Points)	CLO4	Arrived on time, fully prepared, and used time well. Maintained clean coding environment (file structure, naming, saving work). Followed all lab policies.	Arrived slightly late (≤5 mins), prepared, used time well, and maintained proper coding environment. Generally followed lab rules.	Arrived late (>5 mins but < 15 mins), were partially prepared. Completed work but showed little interest. Needed reminders about coding practices/lab discipline.	Arrived very late (≥15 min), were unprepared. Did not focus on lab work, disrupted class, or ignored lab rules.			
TOTAL		<-----Total points earned (Max 100) = Lab Performance Grade >						



Programming Fundamentals Lab Manual



List of Experiments

Exp No.	Title
1	Installation of Ubuntu and Introduction to Simple C programs and Escape Sequences.
2	Variables and implementation of Arithmetic Operations in C
3	Introduction to Control Statements
4	Introduction to Loop Structures
5	Introduction to Functions in C
6	Introduction to Recursion in C
7	Single-Dimensional Arrays in C
8	Multi-Dimensional Arrays in C
	Mid-Semester Exam
9	Pointers and Dynamic Memory Allocation in C-I
10	Pointers and Dynamic Memory Allocation in C-II
11	Introduction to Custom Data Types in C-I
12	Introduction to Custom Data Types in C-II
13	File Processing using C
14	Memory Management
15	OPEN-ENDED LAB-I
16	OPEN-ENDED LAB-II



Programming Fundamentals Lab Manual



Experiment no. 1:

Installation of Ubuntu and Introduction to Simple C programs and Escape Sequences.

Environment Setup Objectives:

By the end of this lab, students will be able to:

1. Installation of Ubuntu Linux.
2. Basic usage of Linux Terminal.
3. Writing, compiling, and executing a simple C program.
4. Understand the purpose and syntax of the printf function in C.
5. Use escape sequences to control program output (newlines, tabs, quotes, etc.).
6. Format text output using conversion specifiers (e.g., %d, %f, %s).
7. Apply multiple printf statements to produce structured and aligned output.
8. Demonstrate proper commenting and code style as recommended in *P.J. Deitel & H.M. Deitel – C How to Program*.
9. Recognize the role of printf in debugging and user interaction within console applications.

Pre-Lab Requirements:

- Windows 10 or later operating system.
- Administrative privileges on the computer.
- Stable internet connection (minimum 2 Mbps).
- At least 2GB free disk space.
- Basic knowledge of C program structure (#include, main, return 0;).
- Familiarity with using Ubuntu Linux.
- Ability to create and execute a simple C source file (.c).
- Review **Chapter 2 (Introduction to C Programming)** from *Deitel & Deitel* for examples on printf.



Programming Fundamentals Lab Manual



Part A: Ubuntu Installation

Step 1: Downloading Ubuntu Setup

- **Ubuntu Desktop Installation Guide (Official):**
 1. **Open** your web browser (Chrome, Firefox, or Edge).
 2. **Navigate** to: <https://documentation.ubuntu.com/desktop/en/latest/tutorial/install-ubuntu-desktop/>.
 3. **Click** on "Download" and **wait** for the download to complete.
 4. **Locate** the downloaded file (usually in Downloads folder).
- **WSL vs Dual Boot (Comparison Guide):**

This guide helps students understand the difference between WSL and Dual Boot and decide which option is more suitable. It also provides guidelines for both the installations.

<https://talhaavvaz.medium.com/wsl-vs-dual-boot-which-is-better-1e317b0cc107>

Method 1: Installing Ubuntu on VirtualBox

This method is recommended for beginners and for students who do not want to modify their existing operating system.

Recommended YouTube Tutorial: Installing Ubuntu on VirtualBox
<https://www.youtube.com/watch?v=x5MhYdijWmc>

Method 2: Dual Boot Ubuntu with Windows

This method allows Ubuntu to be installed alongside Windows on the same system.

Recommended YouTube Tutorial: Dual Boot Ubuntu with Windows
<https://www.youtube.com/watch?v=mXvN1aJYefc>

Important Notes

- Ensure at least 20 GB of free disk space before installation.
- Backup important data before proceeding with Dual Boot installation.
- VirtualBox users must install both VirtualBox and its Extension Pack.



Programming Fundamentals Lab Manual



Part B: Introduction to Terminal

After successful installation of Ubuntu:

1. **Log in** to Ubuntu.
2. **Open** the Terminal using:
 - **Ctrl + Alt + T**
 - OR search **Terminal** from the Applications menu.

Part C: Creating Your First C Program

Step 1: Install GCC Compiler

Before running C programs, install the GCC compiler using the following commands:

```
sudo apt update  
sudo apt install gcc
```

Verify the installation:

```
gcc -- version
```

Step 2: Creating a Workspace

1. **Create** a new folder on Desktop called "PF_Labs"
2. **Inside PF_Labs**, create folder "Lab01_your_roll_number" e.g. Lab01_25_EE_01.
3. **Create** all your tasks files for one lab in its particular folder.
4. **Create** a new file named myfirstCprogram.c:

```
cd Desktop/PF_labs/Lab01_25_EE_01/  
nano myfirstCprogram.c
```



Programming Fundamentals Lab Manual



Step 3: Writing Your First Program

1. Type the following code exactly in editor:

```
#include <stdio.h>

int main()
{
    printf("My First C Program!\n");
    printf("Welcome to Programming Fundamentals!\n");
    return 0;
}
```

2. Save the file with **Ctrl + O**
3. Exit using **Ctrl+X**

Step 3: Compiling and Running Your Program

Type the following commands and compile using GCC.

```
gcc myfirstCprogram.c -o myfirstCprogram
```

Execute the compiled program:

```
./myfirstCprogram
```

Step 4: Insert Your Output (must having your .c file directory)

After completing your program and ensuring it compiles successfully without errors, capture your program output along with the source file location. This step validates both **code execution** and **directory management skills**.

Instructions:

1. Ensure your C source file is properly saved in the designated lab directory.
Example directory path: `C:\Desktop\PF_Labs\Lab01_25_EE_01\myfirstCprogram.c`



Programming Fundamentals Lab Manual



2. Compile your program using **GCC** (or any approved compiler).
Example command: `gcc myfirstCprogram.c -o myfirstCprogram`
3. Run the executable and capture the **complete terminal screenshot** showing:
 - The compilation command.
 - The execution command.
 - The program's output.
 - The full path of your .c source file.

Insert your code screenshot:

Insert your Output screenshot:



Programming Fundamentals Lab Manual



Task 1:

Procedure (step-by-step):

1. Create a file named task1.c.
2. Write a simple program that prints a greeting and demonstrates escapes.
3. Compile and run; fix any syntax errors reported by the compiler.

Sample code:

```
#include <stdio.h>
int main(void)
{
    printf("Hello, Programming Fundamentals!\n");
    printf("This is my First C program\n");
    return 0;
}
```

Output:

```
[Running] cd "d:\PF\LAB-01\" && gcc task1.c -o task1 && "d:\PF\LAB-01\"task1
Hello, Programming Fundamentals!
This is my First C program

[Done] exited with code=0 in 0.456 seconds
```

Student's Code:



Programming Fundamentals Lab Manual

Student's Output:

Task 2:

Write a code to get your name, roll number and section as the output.

Student's Code:



Programming Fundamentals Lab Manual

Student's Output:

Introduction to Escape Sequences:

Escape sequences are special character combinations in C that represent actions or characters that are otherwise difficult to express directly. They always begin with a backslash (\).

Common uses:

- Formatting output (new lines, tabs)
- Printing special characters (quotes, backslash, percent sign)
- Controlling text display (backspace, carriage return, alert).

Following table 1 represents some **common C escape sequences** with their meanings and examples.

Table 1: Common C Escape Sequences

Escape Sequence	Meaning	Example Code	Expected Output (ANSI/Linux)	Windows CMD / PowerShell Output
\n	New line	printf("Hello\nWorld");	Hello World	Hello World
\t	Horizontal tab	printf("A\tB\tC");	A B C (tab spacing)	A B C (tab spacing)
\\	Backslash	printf("\\");	\	\
\"	Double quote	printf("\"Hello\"");	"Hello"	"Hello"
\'	Single quote	printf("'A'");	'A'	'A'
\?	Question mark	printf("\?");	?	?



Programming Fundamentals Lab Manual

	(avoid trigraphs)			
%%	Percent sign	<code>printf("100%%");</code>	100%	100%
\a	Alert (beep)	<code>printf("Beep!\a");</code>	Beep! + system beep	Often just prints Beep! (no sound on most systems)
\b	Backspace	<code>printf("AB\bC");</code>	AC (B erased)	AC (B erased)
\r	Carriage return	<code>printf("Hello\rBye");</code>	Byelo (overwrites Hello)	Byelo (overwrites Hello)
\v	Vertical tab (ASCII 11)	<code>printf("A\vB\n");</code>	A B	A♂B
\f	Form feed (ASCII 12)	<code>printf("Line1\fLine2\n");</code>	Line2 (Line1 cleared or page break)	Line1♀Line2
\0	Null character (string terminator)	<code>printf("Hi\0Hello");</code>	Hi (string ends at \0)	Hi (string ends at \0)

Task 3:

Use multiple printf statements to make hourglass consisting of 7 rows as shown below:

```

*****
*****
***
*
***
*****
*****

```



Programming Fundamentals Lab Manual

Student's Code:

Student's Output:



Programming Fundamentals Lab Manual



Task 4:

Following program prints examples of the most common escape sequences. Students should compile and run it, then observe the behavior.

Sample code:

```
#include <stdio.h>

int main(void)
{
    printf("Hello\nWorld\n");
    printf("A\tB\tC\n");
    printf("\\\n");
    printf("\"Hello\"");
    printf("100%% complete\n");
    printf("AB\bC\n");
    printf("Hello\rBye\n");
    printf("Beep!\a\n");
    printf("A\vB\n");
    printf("Line1\nLine2\n");
    printf("Hi\0Hello\n");
    return 0;
}
```

Student's Output:



Programming Fundamentals Lab Manual



Extensions: (Bonus marks)

1. Write each task 1, task 2 and task 3 code using single printf statement.
2. Print a table of three columns (Name, Roll No, Marks) using \t for spacing.
3. Write a C program that prints the following using **escape sequences**:
 - A message with a **double quote**, a **backslash**, and a **percent sign**.
 - A word where **characters are erased** using \b.
 - A word overwritten by another using \r.
 - Output that demonstrates **alert (\a)**, **form feed (\f)**, and **vertical tab (\v)**.
 - Properly formatted text using **newline (\n)** and **tab (\t)**.
 - Add comments in your code to explain each escape sequence before it is used.
 - Run the program, compare the actual output with your predictions, and record your observations.
4. Use multiple printf statements to get the output as shown below. Pay special attention to the alignment and position of each text and each hourglass at the output. The integers “1234...0123” will help you to align and position the texts and the hourglasses properly. Also pay attention to the number of rows and the number of asterisks (*) in each row of each hourglass. (**Student who use minimum number of printf statement will be given bonus marks**).

123456789012345678	
Hourglass	Hourglass
1	2

***	***
*	*
***	***



Experiment No. 2:

Variables and Implementation of Arithmetic Operations in C

Objectives

The objectives of this lab are:

1. To understand the use of variables.
2. To learn about different data types.
3. To learn how to take input from the user.
4. To implement arithmetic operations in C.
5. To understand the significance of in-built libraries; e.g., math.h.

Prerequisites

Following are the prerequisites of this lab are:

- Installed Ubuntu.
- Basic knowledge of writing simple program in C language.

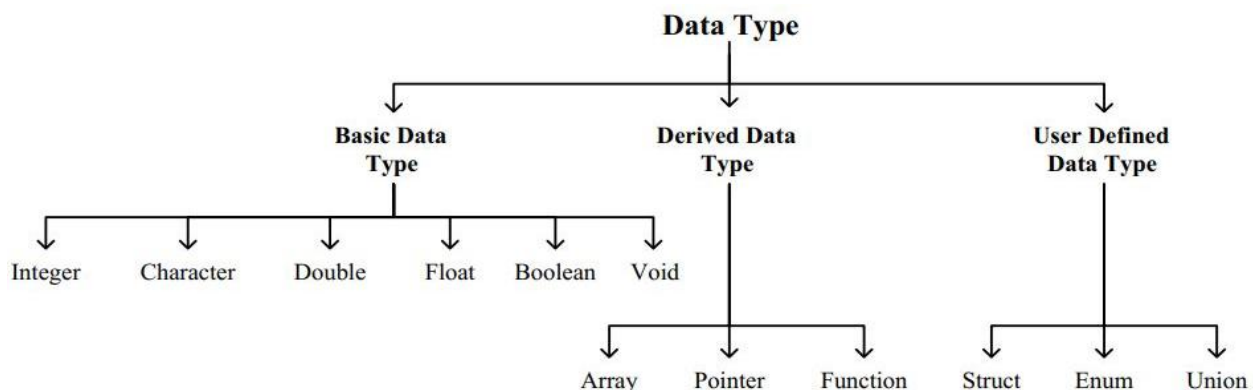
Introduction

Variable

A labeled storage in the memory of the computer, which stores a value is called a variable. It is named variable, because it can be updated during the execution of a program.

Data type

The data type of a variable simply describes which kind of data is stored by that variable. Moreover, data type also determines which operation is going to be performed on that data. It is classified as:





Programming Fundamentals Lab Manual



The focus of this lab is the utilization of basic data types of variables.

Format Specifier

In programming, each data type requires a unique format specifier to be printed after the execution of the statement. Table 1 presents the list of format specifiers.

Table 1: List of format specifiers

Data Type	Key Words in C	Format Specifier
Integer (signed)	int	%d or %i
Unsigned integer	unsigned int	%u
Character	char	%c
Float	float	%f
Double	double	%lf
Boolean	bool	%d or %i
String (array of character)	char str[]	%s

Example

```
#include <stdio.h>
int main() {
    int rollnumber = 20;
    float aggregate = 80.75;
    char section = 'C';
    printf("Age: %d\n", rollnumber);
    printf("GPA: %.2f\n", aggregate);
    printf("Grade: %c\n", section);
    return 0;
}
```

Input-Output Functions

- **scanf Function**
“scanf()” function is used to take input from the keyboard in C language programming.
- **printf Function**
“printf()” function is used to print the output of the executed statement of the C program.

Example

```
#include <stdio.h>
int main() {
    int age;
    printf("Enter your age: ");
}
```



Programming Fundamentals Lab Manual

```
scanf("%d", &age); // '&' address operator
printf("You age is: %d\n", age);
return 0;
}
```

Note: “&” is an address operator. When it is combined with the name of variable it provides the address of that stored variable in the memory.

Comparison of %d and %i Format Specifiers in scanf

Code

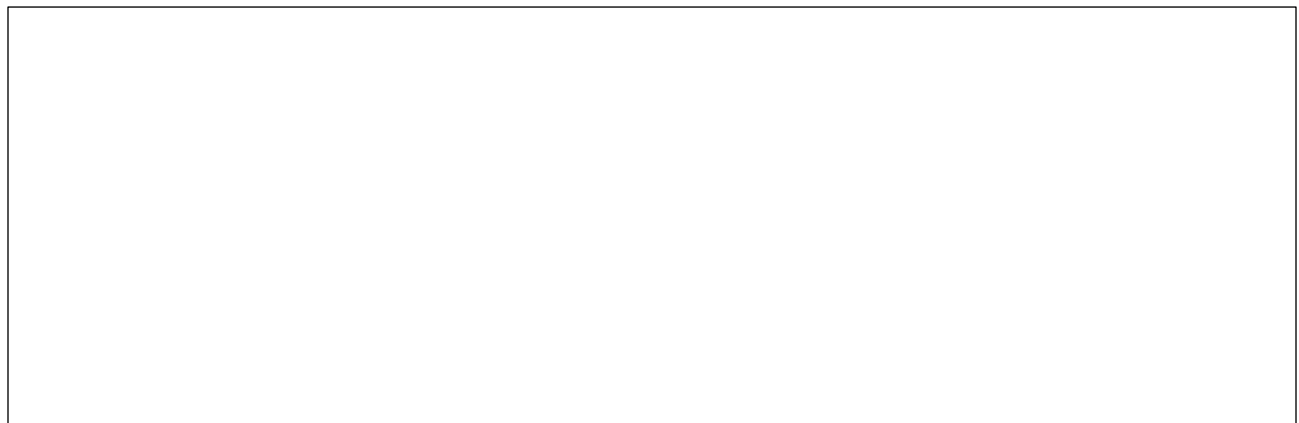
```
#include <stdio.h>

int main() {
    int value;
    printf("Enter a number: ");
    scanf("%i", &value); /*change to %d or %i before running to observe the
behavior of %i and %d for scanf*/

    printf("Value stored (decimal): %d\n", value);
    return 0;
}
```

Note: When “%i” is used in a printf statement, it works exactly like “%d”. However, in a scanf statement, “%i” automatically detects the base (number system) of the input.

Output





Programming Fundamentals Lab Manual



Arithmetic Operations

In the C language, arithmetic operations are performed using arithmetic operators that are mentioned in the following table. Arithmetic operations in C language follow the rule of operator precedence, which are aligned with BODMAS /PEMDAS rules in mathematics.

Table 3: Arithmetic Operations

Operations	Arithmetic Operators	Algebraic Expression	C Expression
Addition	+	$x + y$	$x + b$
Subtraction	-	$a - b$	$a - b$
Multiplication	*	cy	$c * y$
Division	/	$\frac{a}{b}$	a/b
Remainder	%	$x \bmod y$	$x \% y$

Example

```
#include <stdio.h>
int main() {
    int a;
    int b;
    printf("First integer: ");
    scanf("%d", &a);
    printf("Second integer: ");
    scanf("%d", &b);
    printf("Addition: %d\n", a+b);
    printf("Subtraction: %d\n", a-b);
    printf("Multiplication: %d\n", a*b);
    printf("Division: %d\n", a/b);
    printf("Modulus: %d\n", a%b);

    return 0;
}
```

Introduction to <math.h> Header File

In C language, there is a standard library that provides in-built functions for advanced mathematical calculations. The square root of a number can be directly found by including <math.h> header file and calling the in-built function, as shown in the following code.

Example

```
#include <stdio.h>
#include <math.h>
```



Programming Fundamentals Lab Manual



```
int main() {  
    double result = sqrt(144);  
  
    printf("Square root of 144 is: %lf\n", result);  
    return 0;  
}
```

Lab Tasks

Task 1

Find the area and perimeter of a rectangle. Take the measurements of sides of rectangle from the user.

Code



Programming Fundamentals Lab Manual



Output

Task 2

Fix all the syntax and logical errors in the following code, and write the correct version of this C program with its output.

Bugged Code

```
#include <stdio.h>
int main() {
    int a b;
    printf("Enter first number: ")

    scanf("%d", a)

    printf("Enter second number: ");
    scanf("%d", b)

    printf("Addition = %d\n", a + b)
    printf("Subtraction = %f\n", a - b);
    printf("Multiplication = %d\n", a * b)
    printf("Division = %d\n", a / b);

    return;
```



Programming Fundamentals Lab Manual



Corrected Code

Output



Programming Fundamentals Lab Manual



Task 3

Write a C program to swap values of two variables by using a third variable.

Code

Output



Programming Fundamentals Lab Manual



Task 4

Write a C program to find the distance between two points by using in-built functions of math.h library.

Hint: Lets P1= (x1, y1) and P2= (x2, y2). Then, find $D = \sqrt{(x2 - x1)^2 + (y2 - y1)^2}$. You can use in-built functions for square root and power.

Code

Output



Programming Fundamentals Lab Manual



Extension

Bonus Task 1

Write a program to evaluate the following expression when $a=10$, $b=5$, $c=15$, $d=20$ and $\theta = 45^\circ$. The program should work correctly for any values of the variables. Don't forget to include the header `<math.h>` to use trigonometric and `pow()` functions. Use $22/7$ as value of π . The output should be as shown below.

$$\frac{\sqrt{\left(\frac{a}{b} \sin \theta\right)^{\left(\frac{a}{b^2}\right)} + \left(\frac{b}{a} \cos \theta\right)^{\left(\frac{1}{a+b}\right)}}}{(c * \tan \theta) - \left(\frac{d}{c}\right)}$$

Bonus Task 2

Write a C program to calculate power of an number without using `pow()` functions.



Programming Fundamentals Lab Manual



Experiment No. 3:

Introduction to Control Statements

Objectives

The objectives of this lab are:

1. To learn the use of if and if-else statements.
2. To explore nested if-else for multiple conditions.
3. To use the conditional operator (?:) for compact decision-making.
4. To learn how to use logical operators (&&, ||, !) in C.
5. To learn the syntax and usage of the switch statement.
6. To compare if-else and switch statements for multiple conditions.

Prerequisites

Following are the prerequisites of this lab are:

- Basic knowledge of writing simple programs in C language.
- Understanding of variables and arithmetic operations in C.

Introduction

Decision Making

In C programming, decision making is required when different actions need to be performed based on certain conditions. Normally, program statements are executed sequentially, but using decision-making statements we can control the flow of execution.

Selection Statements in C

C provides four types of selection structures in the form of statements.

if Statement (Single-Selection Statement)

- Executes a block of code if a condition is true.
- If the condition is false, the statement is skipped.



Programming Fundamentals Lab Manual



if-else Statement (Double-Selection Statement)

- Executes one block of code if the condition is true.
- Executes another block of code if the condition is false.

Nested if-else Statements

- Nested if-else statements are used to test multiple conditions by placing if-else statements inside another if or else block.
- They are helpful when decisions depend on multiple conditions.

Switch Statement (Multiple-Selection Statement)

- Allows one block of code to be executed out of many alternatives.
- The selection depends on the value of an expression.

Example: if-Statement

```
#include <stdio.h>
int main() {
    int grade = 75;

    if (grade >= 60) {
        printf("Passed\n");
    }
    return 0;
}
```

Example: if-else Statement

```
#include <stdio.h>
int main() {
    int grade = 55;

    if (grade >= 60) {
        printf("Passed\n");
    }
    else {
        printf("Failed\n");
    }
    return 0;
}
```



Programming Fundamentals Lab Manual



Conditional Operator (?:)

C provides the conditional operator (?:), which is closely related to the if-else statement.

- It is the only ternary operator in C (works with three operands).
- Together, these operands form a conditional expression.
- The first operand is the condition to be evaluated.
- The second operand is the value (or expression) returned if the condition is true.
- The third operand is the value (or expression) returned if the condition is false.

Example: Using Conditional Operator

```
#include <stdio.h>
int main() {
    int grade = 55;

    (grade >= 60) ? printf("Passed\n") : printf("Failed\n");

    return 0;
}
```

Note: Use expressions of the same type for the second and third operands of the conditional operator (?:) to avoid subtle errors.

Output





Programming Fundamentals Lab Manual



Example: Nested if-else Statements

```
#include <stdio.h>
int main() {
    int grade;
    char result;
    printf("Enter your grade: ");
    scanf("%d", &grade);

    if (grade >= 90) {
        result = 'A';
    }
    else if (grade >= 80) {
        result = 'B';
    }
    else if (grade >= 70) {
        result = 'C';
    }
    else if (grade >= 60) {
        result = 'D';
    }
    else {
        result = 'F';
    }
    printf("Your grade is: %c\n", result);
    return 0;
}
```

Logical Operators in C

In C, simple conditions such as `counter <= 10`, `total > 1000`, or `number != sentinel Value` are expressed using relational operators (`>`, `<`, `>=`, `<=`) and equality operators (`==`, `!=`). These allow checking only one condition at a time. If multiple conditions are needed, separate statements or nested if/else blocks are required.

To simplify this, C provides logical operators, which allow combining simple conditions into more complex ones:

- **Logical AND (&&)** – True only if *both* conditions are true.
- **Logical OR (||)** – True if *at least one* condition is true.
- **Logical NOT (!)** – Reverses the result of a condition.



Programming Fundamentals Lab Manual



All relational, equality, and logical expressions evaluate to **0 (false)** or **1 (true)**, though any nonzero value is treated as true. Operator precedence also affects evaluation order: for example, **!** has higher precedence than **==**, and both **==** and **>=** are evaluated before **&&**. Logical operators help express multiple conditions compactly and avoid lengthy nested if statements.

Example

```
int main() {
    int marks, age, feePaid;
    marks = 82;
    age = 19;
    feePaid = 0;

    if (marks >= 70 && age >= 18) {
        printf("Eligible based on marks and age.\n");
    }

    if (marks >= 85 || feePaid == 1) {
        printf("Admission confirmed (either high marks OR fee paid).\n");
    }

    if (!(feePaid == 1)) {
        printf("Fee is still pending.\n");
    }

    return 0;
}
```

Compound Statement

The if selection statement expects only one statement in its body, if you have only one statement in the if's body, you do not need to enclose it in braces. To include several statements in the body of an if, you must enclose the set of statements in braces (**{** and **}**). A set of statements contained within a pair of braces is called a compound statement or a block.



Programming Fundamentals Lab Manual



Code

```
#include <stdio.h>

int main() {
    int temperature;

    printf("Enter the temperature in Celsius: ");
    scanf("%d", &temperature);

    if (temperature >= 30) {
        printf("It's a hot day.\n");
    }
    else {
        printf("It's a cool day.\n");
        printf("You might need a jacket.\n");
    }

    return 0;
}
```

Output



Programming Fundamentals Lab Manual



Example: Switch Statement

```
#include <stdio.h>

int main() {
    char SHAPE;
    float RADIUS, SIDE, LENGTH, WIDTH, BASE, HEIGHT;

    printf("Enter shape (Circle/Square/Rectangle/Triangle): ");
    scanf(" %c", &SHAPE);

    switch (SHAPE) {
        case 'C':
            printf("Enter radius: ");
            scanf("%f", &RADIUS);
            printf("Area of Circle = %.2f\n", 3.14159 * RADIUS * RADIUS);
            break;
        case 'S':
            printf("Enter side: ");
            scanf("%f", &SIDE);
            printf("Area of Square = %.2f\n", SIDE * SIDE);
            break;
        case 'R':
            printf("Enter length and width: ");
            scanf("%f %f", &LENGTH, &WIDTH);
            printf("Area of Rectangle = %.2f\n", LENGTH * WIDTH);
            break;
        case 'T':
            printf("Enter base and height: ");
            scanf("%f %f", &BASE, &HEIGHT);
            {
                float AREA = 0.5 * BASE * HEIGHT;
                printf("Area of Triangle = %.2f\n", AREA);
            }
            break;
        default:
            printf("Invalid selection.\n");
    }

    return 0;
}
```



Programming Fundamentals Lab Manual



Output

Lab Tasks

Task 1

Write a program which takes three integers from the user and displays the maximum integer of the three.



Programming Fundamentals Lab Manual



Output

Task 2

Complete the following code and remove all the errors to get the output as shown below.

Bugged Code

```
printf("Enter your age:");
scanf("%f", age);

if (age >= 18):
{
    printf("You are eligible for the driving license.\n");
}

else:
    print('You are not eligible for the driving license.\n')
}
```



Programming Fundamentals Lab Manual



Corrected Code

Output



Programming Fundamentals Lab Manual



Task 3

Write a C program that takes a year as input from the user and uses nested if to check whether it is a leap year or not.

Code

Output



Programming Fundamentals Lab Manual



Task 4

Write a C program that prompts the user to enter an integer number from 1 to 7 and displays the corresponding day of the week. The program should use a **switch statement** to determine the day. If the user enters a number outside this range, the program should display a message indicating **“Invalid input”**.

Code

Output



Programming Fundamentals Lab Manual



Experiment No. 4:

Introduction to Loop Structures

Objectives

The objectives of this lab are:

1. To understand the structure and control flow of the for loop.
2. To implement counter-controlled repetition using for loops.
3. To understand assignment ($+=$, $-=$) and increment/decrement ($++$, $--$) operators within loops.
4. To practice nested for loops for sequences and pattern generation.
5. To explore the use of break and continue for controlling loop execution.

Prerequisites

Following are the prerequisites of this lab are:

- Knowledge of relational and logical operators for condition checking.
- Familiarity with decision-making statements (if, if-else, nested if-else and switch statements).

Introduction

Most programs involve repetition, or looping. A loop is a group of instructions the computer executes repeatedly while some loop-continuation condition remains true.

Repetition in C

Counter-controlled (Definite repetition)

- Number of repetitions known in advance.
- Uses a control variable, incremented until the required count is reached.

Sentinel-controlled (Indefinite repetition)

- Number of repetitions not known in advance.
- A sentinel value signals “end of data” and must differ from regular data.

Repetition Statements in C

In C, repetition (looping) allows executing a block of code multiple times until a condition is satisfied. There are three main repetition statements:



Programming Fundamentals Lab Manual



for Repetition Statement

- Used when the number of repetitions is known in advance.
- Combines initialization, condition, and increment in one line.

while Repetition Statement

- A pre-test loop: condition is checked before each iteration.
- Suitable when the number of repetitions is not known in advance.

do-while Repetition Statement

- A post-test loop: condition is checked after each iteration.
- Ensures the loop body executes at least once.

The focus of this lab is the utilization of for loops and nested for loops.

Arithmetic Assignment Operators

C provides several assignment operators that allow abbreviating assignment expressions. For example, the statement `c = c + 3;` can be written in the shorter form `c += 3;`. The operator `+=` adds the value of the expression on the right side to the variable on the left side and stores the result in the same variable. In general, any statement written as `variable = variable operator expression;` can be expressed in the form `variable operator= expression;`, where the operator may be `+`, `-`, `*`, `/`, `%`, or others.

Table 1: List of Assignment Operators

Assume: `int c = 3, d = 5, e = 4, f = 6, g = 12;`

Operator	Sample Expression	Equivalent Expression	Result Assigned
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 to c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 to d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 to e
<code>/=</code>	<code>f /= 3</code>	<code>f = f / 3</code>	2 to f
<code>%=</code>	<code>g %= 9</code>	<code>g = g % 9</code>	3 to g

Increment and Decrement Operators

C provides the unary increment (`++`) and decrement (`--`) operators. Preincrement/decrement changes the variable before using it in an expression, while postincrement/decrement uses the current value first, then updates the variable.



Programming Fundamentals Lab Manual



Table 2: List of Increment and Decrement Operators

Operator	Sample Expression	Explanation
++	++a	Increment a by 1, then use the new value in the expression.
++	a++	Use the current value of a in the expression, then increment a by 1.
--	--b	Decrement b by 1, then use the new value in the expression.
--	b--	Use the current value of b in the expression, then decrement b by 1.

Table 3: Statement Header Components

Component	Description	Example
Initialization of control variable	Sets the starting value of the counter	counter = 1
Loop-continuation condition	Condition that must remain true for loop to continue	counter <= 10
Increment of control variable	Updates the counter after each iteration	++counter

Example:

```
#include <stdio.h>

int main(void) {
    unsigned int counter;

    for (counter = 1; counter <= 10; ++counter) {
        printf("Counter value: %u\n", counter);
    }

    return 0;
}
```

Output



Programming Fundamentals Lab Manual



Example:

```
#include <stdio.h>

int main(void) {
    int sum = 0;

    for (int i = 1; i <= 10; i++) {
        sum += i;
    }

    printf("Sum of first 10 natural numbers: %d\n", sum);

    return 0;
}
```

break and continue Statements

The break statement immediately exits a loop or switch, while continue skips the rest of the current iteration and moves to the next. In for loops, the increment executes before checking the condition; in while/do-while, the condition is checked immediately.

Example

```
#include <stdio.h>

int main(void) {
    int x;

    printf("Using break:\n");
    for (x = 1; x <= 10; ++x) {
        if (x == 5) {
            break;
        }
        printf("%d ", x);
    }
    printf("\nBroke out of loop at x = %d\n\n", x);

    printf("Using continue:\n");
    for (x = 1; x <= 10; ++x) {
        if (x % 2 == 0) {
            continue;
        }
        printf("%d ", x);
    }
}
```



Programming Fundamentals Lab Manual



```
        continue;
    }
    printf("%d ", x);
}
printf("\n");

return 0;
}
```

Output



Nested for-loop Statement

A nested for loop is a loop inside another loop. The outer loop controls the number of times the inner loop executes, and the inner loop performs its full cycle for each iteration of the outer loop. Nested loops are commonly used to handle multi-dimensional data or to print patterns.

Example

```
#include <stdio.h>

int main() {
    int i, j;
    int rows = 5;

    for (i = 1; i <= rows; i++) {
        for (j = 1; j <= i; j++) {
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```



Programming Fundamentals Lab Manual



Output

Lab Tasks

Task 1

Write a C program to generate the following pattern using nested for-loops.

```
1
12
123
1234
12345
```

Code



Programming Fundamentals Lab Manual



Output

Task 2

Write a program that prompts the user to enter a positive integer and computes its factorial.

Code



Programming Fundamentals Lab Manual



Output

Task 3

Write a C program that prints all numbers from 1 to 50 using a for loop. The program should skip all multiples of 4 and stop printing entirely if a number greater than 40 is encountered.

Code



Programming Fundamentals Lab Manual



Output

Task 4

Print the following using nested for-loops.

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 5 = 5

2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
2 x 4 = 8
2 x 5 = 10
```

Code



Programming Fundamentals Lab Manual



Output

While and Do-While Repetition Statements in C

In C programming, the while statement performs repetition as long as a condition evaluates to true, making it an entry-controlled loop. The do-while statement, on the other hand, checks its condition after executing the loop body, ensuring that the body executes at least once hence, it is an exit-controlled loop.

This lab focuses on understanding the control flow of while and do-while repetition statements, and their use in implementing counter-controlled and sentinel-controlled loops for solving both definite and indefinite repetition problems.



Programming Fundamentals Lab Manual

Table 4: While Statement Header Components

Component	Description	Counter-Controlled Example	Sentinel-Controlled Example
Initialization	Sets the initial value of the control variable or prepares the first input.	counter = 1; total = 0;	printf("Enter grade (-1 to end): "); scanf("%d", &grade); total = 0;
Loop Continuation Condition	Condition that must remain true for the loop to continue executing.	while (counter <= 10)	while (grade != -1)
Update (Within Loop Body)	Changes the control variable or takes new input to eventually make the condition false.	counter++;	scanf("%d", &grade);

Example:

```
#include <stdio.h>

int main() {
    int product = 3;

    while (product <= 100) {
        printf("Product = %d\n", product);
        product *= 3;
    }

    printf("Final Product = %d\n", product);
    return 0;
}
```

Output



Programming Fundamentals Lab Manual



Example:

```
#include <stdio.h>

int main(void)
{
    unsigned int counter;
    int grade;
    int total;
    float average;

    total = 0;
    counter = 0;

    printf("Enter grade, -1 to end: ");
    scanf("%d", &grade);

    while (grade != -1) {
        total = total + grade;
        counter = counter + 1;

        printf("Enter grade, -1 to end: ");
        scanf("%d", &grade);
    }

    if (counter != 0) {
        average = (float) total / counter;

        printf("Class average is %.2f\n", average);
    }
    else {
        printf("No grades were entered.\n");
    }

    return 0;
}
```



Programming Fundamentals Lab Manual



Output

--

Table 5: do-while Statement Header Components

Component	Description	Example
Initialization of control variable	Sets the starting value before entering the loop.	counter = 1;
Loop body (action)	Contains the statements to execute repeatedly.	printf("%d\n", counter);
Increment / Update of control variable	Updates the loop variable each iteration to eventually make the condition false.	counter++;
Loop-continuation condition	Tested after the loop body; determines whether to repeat the loop.	while (counter <= 10);

Example

```
#include <stdio.h>

int main(void)
{
    unsigned int counter = 1;

    do {
        printf("%u ", counter);
    } while (++counter <= 10);

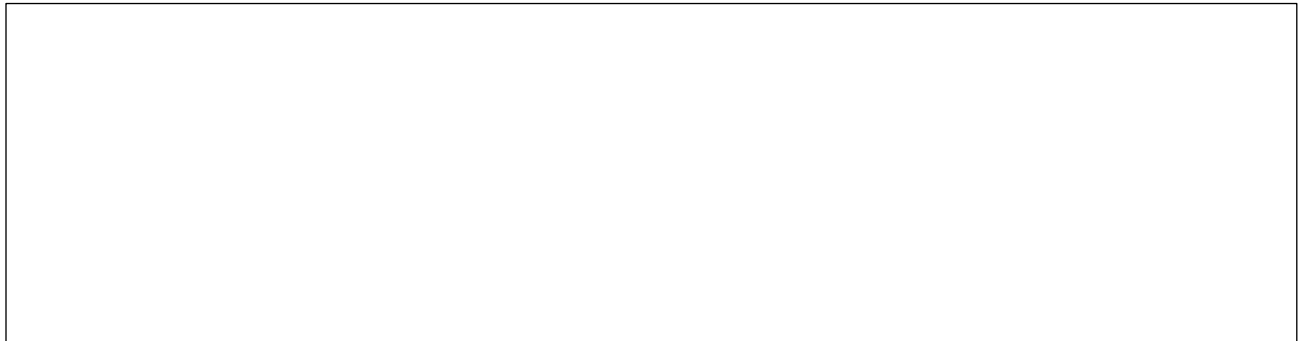
    printf("\n");
    return 0;
}
```



Programming Fundamentals Lab Manual



Output



Infinite loop

Not providing in the body of a while statement an action that eventually causes the condition in the while to become false. Normally, such a repetition structure will never terminate an error called an “infinite loop”.

Example

```
#include <stdio.h>

int main(void)
{
    int z = 100;
    int sum = 0;

    while (z >= 0)
        sum += z;

    // This line will never be reached
    printf("Sum = %d\n", sum);

    return 0;
}
```

Nested Control Statements

Nested control statements occur when one control structure such as a decision-making or looping statement is placed inside another. For example, an if statement inside another if, or a while loop inside a for loop. Nesting allows programs to handle more complex decision-making and repetitive tasks by combining multiple levels of



Programming Fundamentals Lab Manual

logic or iteration. Proper indentation and braces are important in nested structures to make the code clear, readable, and logically organized.

Example

```
#include <stdio.h>

int main(void)
{
    unsigned int row = 10;
    unsigned int column;

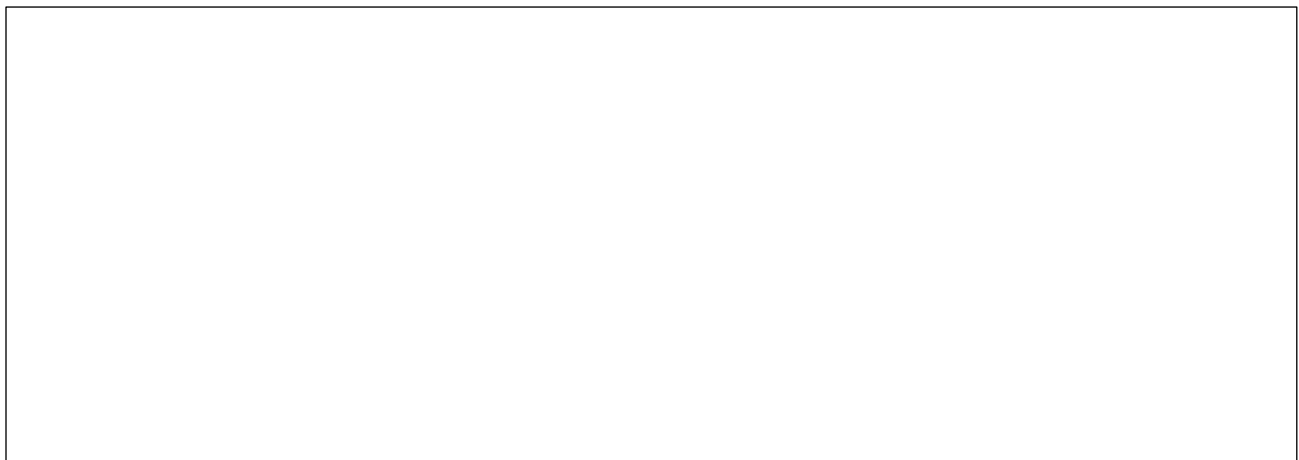
    while (row >= 1) {
        column = 1;

        while (column <= 10) {
            printf("%s", row % 2 ? "<" : ">");
            ++column;
        }

        --row;
        printf("\n");
    }

    return 0;
}
```

Output





Programming Fundamentals Lab Manual



Example

```
#include <stdio.h>

int main(void) {
    int row = 1;

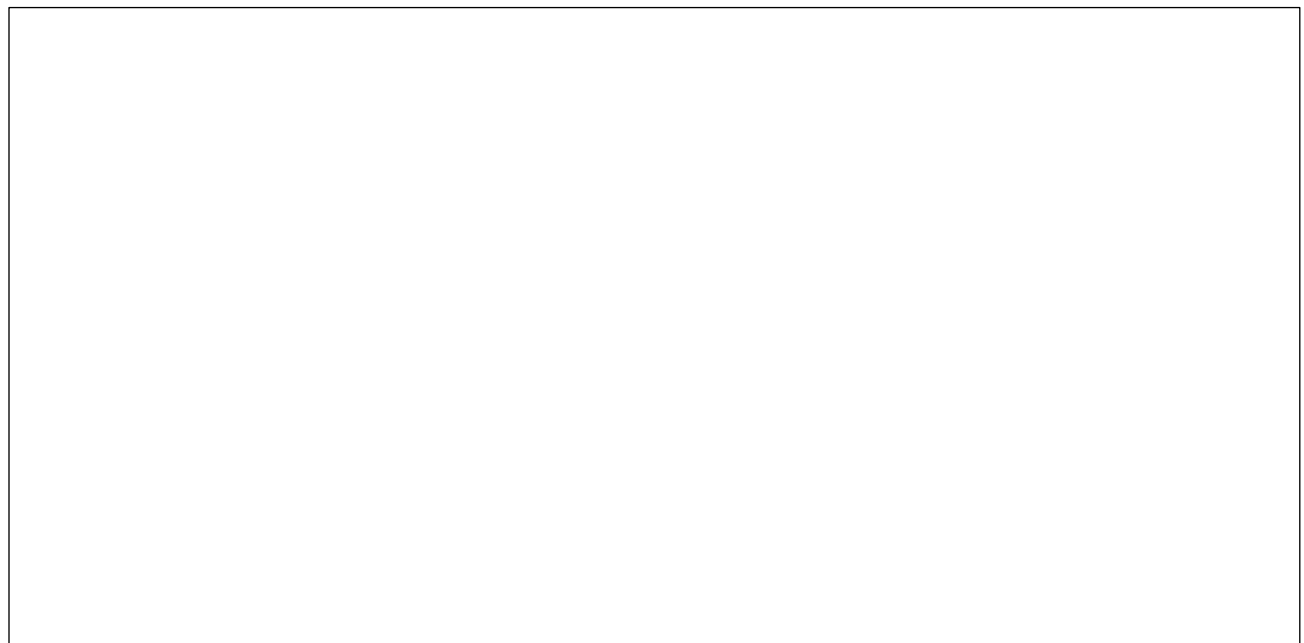
    do {
        int col = 1;

        do {
            printf("* ");
            col++;
        } while (col <= 3);

        printf("\n");
        row++;
    } while (row <= 3);

    return 0;
}
```

Output





Programming Fundamentals Lab Manual

Lab Tasks

Task 5

Write a program that uses while loop to print the following table of values. Use the tab escape sequence, `\t`, in the `printf` statement to separate the columns with tabs.

N	10*N	100*N	1000*N
1	10	100	1000
2	20	200	2000
3	30	300	3000
4	40	400	4000
5	50	500	5000
6	60	600	6000
7	70	700	7000
8	80	800	8000
9	90	900	9000
10	100	1000	10000

Code



Programming Fundamentals Lab Manual

Output

Task 6

Write a C program that uses a do-while loop to calculate and display the average of several integers entered by the user. The input ends when the user enters the sentinel value 9999, which should not be included in the calculation. The program should keep track of the sum and count of valid numbers, then compute and print the average as a floating-point value. If no valid numbers are entered, display a message saying that no numbers were entered.

Code



Programming Fundamentals Lab Manual



Output



Task 7

Fix all the syntax and logical errors in the following code, and write the correct version of this C program with its output.

Bugged Code

```
#include <stdio.h>

int main(void)
{
    unsigned int count = 1;

    while (count <= 10);
    {
        printf(count % 2 ? **** : ++++++++);
        count + 1;
    }

    return 0
```



Programming Fundamentals Lab Manual

Corrected Code

Output

Task 8

Write a program that finds the smallest of several integers. Assume that the first value read specifies the number of values remaining. The program should use a while loop to repeatedly read each number until all values are processed. Inside the loop, use an if-else statement to compare each new number with the current smallest value and update it when a smaller number is found.



Programming Fundamentals Lab Manual



Code

Output